LogosFuzz

AUTOMATED FUZZING PIPELINE
EXT → SCH → GEN → EXE → ANA

소스 코드에서 크래시 리포트까지, LLM이 잇는 자동 퍼징

C/C++ 소스와 Bazel 빌드 그래프를 읽어 퍼징 하네스를 스스로 생성·수선하고, 격리 환경에서 실행한 뒤 크래시의 정탐 여부까지 판별하는 End-to-End 파이프라인

- BAZEL-AWARE
- LIBFUZZER + ASAN/UBSAN/LSAN
- SELF-HEAL LOOP
- HUMAN-IN-THE-LOOP

01 파이프라인 6단계

EXT-01
추출
- Bazel 의존 그래프(deps·include·copts)
- C/C++17 AST 분석
- API 제약조건 → RAG 지식베이스

▶

SCH-02
스케줄링
- 같은 상태를 공유하는 API를 Logic Group으로 패키징
- 시너지 우선순위·자원 할당

▶

GEN-03
하네스 생성
- 제약조건 컨텍스트로 LLM 하네스 초안
- 빌드 에러 분류 → 자가치유
- 입력 소비 정적 검증

EXE-04
격리 퍼징 실행
- Docker 격리 libFuzzer 실행
- 커버리지·크래시 수집
- 타임아웃·코퍼스 관리

▶

ANA-05
크래시 분석
- 중복 제거·근본 원인 분석
- 정탐/오탐 자동 판별
- CVE 리포트·KB 역피드백

▶

CTR-06
제어 · HITL
- 파이프라인 컨트롤러
- 체크포인트별 AUTO / MANUAL / CONDITIONAL 승인 정책

02 핵심 기술

//
Bazel 빌드 그래프 인지
strict deps·visibility 를 강제하는 환경에서 정답을 아는 건 **bazel query** 뿐. 타깃 경계를 넘는 API 는 한 그룹으로 묶지 않습니다.

↻
실측 코퍼스 기반 자가치유
실제로 빌드를 깨뜨려 모은 에러 원문으로 **10종 분류기**를 구성. 고칠 수 없는 결함은 LLM 호출 없이 즉시 에스컬레이션합니다.

✓
검증된 하네스만 실행
퍼즈 입력을 쓰지 않는 하네스는 커버리지만 나오고 새 경로가 없습니다. **입력 소비 검증**으로 헛도는 캠페인을 사전에 차단합니다.

03 자가치유 실제 복구

SelfHealLoop · bazel build //harness:json_fuzzer

```
최종 결과  success  (LLM 수정 라운드 1회)

라운드 0 빌드 실패
  진단 = undefined_symbol / resolve_symbol
  ld.lld: error: undefined symbol:
  score::json::ParseStrict(...)

라운드 1 빌드 성공
  산출물 = bazel-bin/harness/json_fuzzer
```

04 검증 현황

TESTS PASSED
395
EC2 회귀 검증 (skip 4)

S-CORE APIS
3,080
→ 682 Logic Group

DLT APIS → GROUPS
230 →21
제약조건 커버리지 87.4%

LIBFUZZER EXEC/S
518 K
DLT 300초 실행, 크래시 0

github.com/sesezz/LogosFuzz

대상 Eclipse S-CORE baselibs · DLT daemon · Automotive UDS
스택 Python · libclang · Bazel · libFuzzer · Docker